

MODUL PRAKTIKUM KRIPTOGRAFI MODERN



Program Studi Sistem Informasi
Fakultas Ilmu Komputer
Universitas Jember

Tahun Akademik 2025/2026

Kata Pengantar

Modul praktikum Kriptografi Modern ini disusun sebagai panduan bagi mahasiswa dalam melaksanakan kegiatan praktikum pada mata kuliah Kriptografi Modern.

Modul ini dirancang untuk membantu mahasiswa memahami konsep-konsep kriptografi modern melalui kegiatan praktikum berbasis implementasi menggunakan bahasa pemrograman Python.

Materi praktikum meliputi:

- Setup lingkungan pemrograman kriptografi
- Algoritma kriptografi asimetrik
- Fungsi hash
- Digital signature
- Public Key Infrastructure

Diharapkan melalui praktikum ini mahasiswa tidak hanya memahami konsep kriptografi secara teoritis tetapi juga mampu mengimplementasikan sistem kriptografi sederhana secara mandiri.

Jember, March 10, 2026

Penyusun

Contents

1	Kriptografi Tradisional	5
1.1	Pendahuluan	5
1.2	Tujuan Kriptografi Tradisional	5
1.3	Metode Substitusi	5
1.4	Metode Transposisi	6
1.5	Kelemahan Kriptografi Tradisional	6
1.6	Perbandingan dengan Kriptografi Modern	7
2	Praktikum Kriptografi Tradisional	7
2.1	Tujuan Praktikum	7
2.2	Dasar Teori Singkat	7
2.3	Praktikum 1: Caesar Cipher	7
2.4	Praktikum 2: Vigenère Cipher	9
2.5	Analisis dan Diskusi	10
2.6	Tabel Hasil Percobaan	10
2.7	Kesimpulan	11
2.8	Output yang Dikumpulkan	11
3	Praktikum RSA	11
3.1	Implementasi Algoritma RSA	11
3.1.1	Tujuan Praktikum	11
3.1.2	Dasar Teori	11
3.1.3	Praktikum 2A – Generate Kunci RSA	12
3.1.4	Praktikum 2B – Enkripsi dan Dekripsi	12
3.1.5	Praktikum 2C – Pengukuran Waktu Komputasi	13
4	Tugas Praktikum RSA	14
4.1	Instruksi Tugas	14
4.2	Eksperimen yang Dilakukan	14
4.3	Tabel Hasil Percobaan	15
4.4	Pertanyaan Analisis	15
4.5	Output yang Dikumpulkan	15

Pendahuluan

Kriptografi modern merupakan bidang ilmu yang mempelajari teknik pengamanan informasi dengan menggunakan metode matematis dan algoritma komputasi.

Dalam praktikum ini mahasiswa akan mempelajari implementasi algoritma kriptografi modern menggunakan bahasa pemrograman Python dan berbagai library kriptografi.

Praktikum dirancang berbasis eksperimen sehingga mahasiswa dapat memahami karakteristik algoritma kriptografi melalui pengukuran dan analisis.

Peraturan Praktikum

Mahasiswa wajib:

1. Membawa laptop sendiri
2. Menginstal Python sebelum praktikum
3. Membaca modul sebelum praktikum
4. Mengumpulkan laporan praktikum tepat waktu
5. Mengikuti praktikum secara mandiri

Format Laporan

Laporan praktikum harus berisi:

1. Judul Praktikum
2. Tujuan Praktikum
3. Source Code
4. Hasil Percobaan
5. Analisis
6. Kesimpulan

1 Kriptografi Tradisional

1.1 Pendahuluan

Kriptografi tradisional merupakan metode pengamanan pesan yang telah digunakan sejak zaman kuno sebelum berkembangnya kriptografi modern berbasis teori bilangan dan komputasi.

Kriptografi tradisional umumnya menggunakan teknik:

- **Substitusi** (penggantian huruf)
- **Transposisi** (perubahan posisi huruf)

Metode ini menggunakan satu kunci yang sama untuk enkripsi dan dekripsi, sehingga termasuk ke dalam kategori *symmetric cryptography*.

1.2 Tujuan Kriptografi Tradisional

Tujuan utama kriptografi tradisional adalah:

1. Menyembunyikan isi pesan (confidentiality)
2. Mengamankan komunikasi militer dan diplomatik
3. Menghindari pembacaan pesan oleh pihak yang tidak berwenang

Namun, kriptografi tradisional belum mampu menjamin:

- Integrity secara kuat
- Authenticity
- Non-repudiation

1.3 Metode Substitusi

Metode substitusi mengganti setiap huruf plaintext dengan huruf lain.

Contoh paling sederhana adalah **Caesar Cipher**.

Jika pergeseran kunci adalah $k = 3$, maka:

$$C = (P + 3) \mod 26$$

di mana:

- P = posisi huruf plaintext
- C = posisi huruf ciphertext

Contoh

Plaintext: KRIPTO Ciphertext: NULSWR

1.4 Metode Transposisi

Metode transposisi tidak mengganti huruf, tetapi mengubah posisi huruf.

Contoh sederhana adalah *Rail Fence Cipher*.

Plaintext:

K R I P T O G R A F I

Dengan 2 baris:

K I T G A I

R P O R F

Ciphertext:

K I T G A I R P O R F

1.5 Kelemahan Kriptografi Tradisional

Beberapa kelemahan utama:

1. Ruang kunci kecil
2. Rentan terhadap *frequency analysis*
3. Tidak tahan terhadap komputasi modern
4. Tidak cocok untuk sistem digital

Sebagai contoh, Caesar Cipher hanya memiliki 25 kemungkinan kunci, sehingga dapat dipecahkan dengan *brute force* dalam waktu singkat.

1.6 Perbandingan dengan Kriptografi Modern

Aspek	Tradisional	Modern
Dasar	Pergeseran huruf	Teori bilangan
Keamanan	Rendah	Tinggi
Ruang kunci	Kecil	Sangat besar
Cocok untuk	Manual	Sistem digital

2 Praktikum Kriptografi Tradisional

2.1 Tujuan Praktikum

Setelah mengikuti praktikum ini mahasiswa mampu:

1. Mengimplementasikan algoritma kriptografi tradisional
2. Memahami konsep substitusi dan transposisi
3. Melakukan enkripsi dan dekripsi manual
4. Menganalisis kelemahan kriptografi klasik

2.2 Dasar Teori Singkat

Kriptografi tradisional menggunakan teknik dasar:

- Substitusi (penggantian huruf)
- Transposisi (perubahan posisi huruf)

Pada praktikum ini akan diimplementasikan:

1. Caesar Cipher
2. Vigenère Cipher

2.3 Praktikum 1: Caesar Cipher

Rumus

$$C = (P + k) \mod 26$$

$$P = (C - k) \mod 26$$

dengan:

- P = plaintext
- C = ciphertext
- k = kunci pergeseran

Implementasi Python

```
def caesar_encrypt(plaintext, key):
    result = ""
    for char in plaintext:
        if char.isalpha():
            shift = ord('A')
            result += chr((ord(char) - shift + key) % 26 + shift)
        else:
            result += char
    return result

def caesar_decrypt(ciphertext, key):
    return caesar_encrypt(ciphertext, -key)

pesan = "KRIPTOGRAFI"
kunci = 3

cipher = caesar_encrypt(pesan, kunci)
plain = caesar_decrypt(cipher, kunci)

print("Ciphertext:", cipher)
print("Decrypted :", plain)
```

Tugas

1. Ubah nilai kunci menjadi 5 dan 10
2. Catat hasil ciphertext

3. Lakukan brute force untuk semua kemungkinan kunci (1–25)

2.4 Praktikum 2: Vigenère Cipher

Konsep

Vigenère Cipher menggunakan kunci berupa kata.

Jika kunci adalah “KEY”, maka setiap huruf plaintext digeser berdasarkan huruf kunci yang bersesuaian.

Implementasi Python

```
def vigenere_encrypt(plaintext, key):
    result = ""
    key = key.upper()
    key_index = 0

    for char in plaintext:
        if char.isalpha():
            shift = ord('A')
            k = ord(key[key_index % len(key)]) - shift
            result += chr((ord(char) - shift + k) % 26 + shift)
            key_index += 1
        else:
            result += char
    return result

def vigenere_decrypt(ciphertext, key):
    result = ""
    key = key.upper()
    key_index = 0

    for char in ciphertext:
        if char.isalpha():
            shift = ord('A')
            k = ord(key[key_index % len(key)]) - shift
            result += chr((ord(char) - shift - k) % 26 + shift)
```

```

        key_index += 1
    else:
        result += char
    return result

pesan = "KRIPTOGRAFI"
kunci = "KEY"

cipher = vigenere_encrypt(pesan, kunci)
plain = vigenere_decrypt(cipher, kunci)

print("Ciphertext:", cipher)
print("Decrypted :", plain)

```

Tugas

1. Gunakan kunci berbeda (contoh: DATA, SECURE)
2. Bandingkan hasilnya dengan Caesar Cipher
3. Jelaskan mengapa Vigenère lebih kuat dibanding Caesar

2.5 Analisis dan Diskusi

Mahasiswa wajib menjawab:

1. Mengapa Caesar Cipher sangat mudah dipecahkan?
2. Apa yang dimaksud dengan frequency analysis?
3. Mengapa panjang kunci mempengaruhi keamanan Vigenère?
4. Apakah metode ini aman digunakan pada sistem digital modern?

2.6 Tabel Hasil Percobaan

Metode	Kunci	Ciphertext
Caesar	3	
Caesar	5	
Vigenère	KEY	
Vigenère	DATA	

2.7 Kesimpulan

Tuliskan kesimpulan mengenai:

- Kelemahan kriptografi tradisional
- Perbedaan substitusi tunggal dan substitusi polialfabetik
- Alasan kriptografi tradisional tidak digunakan lagi dalam sistem keamanan modern

2.8 Output yang Dikumpulkan

- Source code
- Screenshot hasil eksekusi
- Jawaban analisis
- Tabel hasil percobaan

3 Praktikum RSA

3.1 Implementasi Algoritma RSA

3.1.1 Tujuan Praktikum

Setelah mengikuti praktikum ini mahasiswa mampu:

1. Membuat pasangan kunci RSA
2. Melakukan enkripsi dan dekripsi RSA
3. Membandingkan ukuran kunci RSA
4. Mengukur waktu komputasi RSA

3.1.2 Dasar Teori

RSA merupakan algoritma kriptografi kunci publik yang berbasis pada kesulitan faktorisasi bilangan besar.

RSA menggunakan dua kunci:

- Public key untuk enkripsi
- Private key untuk dekripsi

Proses RSA:

1. Generate pasangan kunci
2. Enkripsi pesan
3. Dekripsi pesan

3.1.3 Praktikum 2A – Generate Kunci RSA

```
from cryptography.hazmat.primitives.asymmetric import rsa
```

```
private_key = rsa.generate_private_key(
    public_exponent=65537,
    key_size=1024
)
```

```
public_key = private_key.public_key()
```

```
print("RSA 1024-bit berhasil dibuat")
```

Ubah ukuran kunci menjadi:

- 1024 bit
- 2048 bit

Bandingkan hasilnya.

3.1.4 Praktikum 2B – Enkripsi dan Dekripsi

```
from cryptography.hazmat.primitives.asymmetric import padding
from cryptography.hazmat.primitives import hashes
```

```
pesan = b"Belajar RSA"
```

```
ciphertext = public_key.encrypt(
```

```

pesan,
padding.OAEP(
mgf=padding.MGF1(algorithm=hashes.SHA256()),
algorithm=hashes.SHA256(),
label=None
)
)

```

```

plaintext = private_key.decrypt(
ciphertext,
padding.OAEP(
mgf=padding.MGF1(algorithm=hashes.SHA256()),
algorithm=hashes.SHA256(),
label=None
)
)

```

```

print("Plaintext :", plaintext)

```

Pastikan hasil dekripsi sama dengan pesan awal.

3.1.5 Praktikum 2C – Pengukuran Waktu Komputasi

Program berikut mengukur waktu pembuatan kunci RSA.

```

import time
from cryptography.hazmat.primitives.asymmetric import rsa

start = time.time()

private_key = rsa.generate_private_key(
public_exponent=65537,
key_size=2048
)

end = time.time()

print("Waktu pembuatan kunci:", end-start)

```

Lakukan pengukuran untuk:

- RSA 1024 bit
- RSA 2048 bit

Catat hasilnya.

4 Tugas Praktikum RSA

Pada bagian ini mahasiswa diminta melakukan beberapa eksperimen untuk menganalisis pengaruh ukuran kunci terhadap performa algoritma RSA.

4.1 Instruksi Tugas

Lakukan langkah-langkah berikut:

1. Generate pasangan kunci RSA dengan ukuran **1024 bit**.
2. Generate pasangan kunci RSA dengan ukuran **2048 bit**.
3. Ukur waktu yang dibutuhkan untuk proses pembuatan kunci.
4. Lakukan proses enkripsi pesan menggunakan kedua ukuran kunci.
5. Bandingkan ukuran ciphertext yang dihasilkan.

4.2 Eksperimen yang Dilakukan

Eksperimen	Deskripsi
Eksperimen 1	Generate pasangan kunci RSA dengan ukuran 1024 bit
Eksperimen 2	Generate pasangan kunci RSA dengan ukuran 2048 bit
Eksperimen 3	Bandingkan waktu yang dibutuhkan untuk proses generate kunci
Eksperimen 4	Bandingkan ukuran ciphertext yang dihasilkan dari proses enkripsi

4.3 Tabel Hasil Percobaan

Mahasiswa diminta mencatat hasil eksperimen pada tabel berikut.

Ukuran Kunci	Waktu Generate (detik)	Ukuran Ciphertext (byte)
1024 bit		
2048 bit		

4.4 Pertanyaan Analisis

Jawab pertanyaan berikut berdasarkan hasil eksperimen yang dilakukan.

1. Manakah yang membutuhkan waktu lebih lama untuk proses pembuatan kunci, 1024 bit atau 2048 bit?
2. Mengapa ukuran kunci yang lebih besar meningkatkan keamanan RSA?
3. Bagaimana pengaruh ukuran kunci terhadap waktu komputasi?
4. Apakah ukuran kunci mempengaruhi ukuran ciphertext? Jelaskan.

4.5 Output yang Dikumpulkan

Mahasiswa wajib mengumpulkan:

- Source code program
- Screenshot hasil eksekusi program
- Tabel hasil percobaan
- Jawaban analisis